1. Download Advertising Dataset and Convert to DataFrame

We will download the dataset directly from a public source that hosts the classic Advertising dataset (often used in Introduction to Statistical Learning) to avoid Kaggle API credential issues, and load it into a pandas DataFrame.

```
#MOHANASUNDARAM
#241801169
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# URL to the Advertising dataset
url = 'https://www.statlearning.com/s/Advertising.csv'
df = pd.read_csv(url, index_col=0)

# Display the first few rows
display(df.head())
```

	TV	radio	newspaper	sales
1	230.1	37.8	69.2	22.1
2	44.5	39.3	45.1	10.4
3	17.2	45.9	69.3	9.3
4	151.5	41.3	58.5	18.5
5	180.8	10.8	58.4	12.9

2. Explore and Preprocess Data

We will check the dataset structure, examine null values, define features and target, and create a pair plot.

```
# Check data types and null values
print("Dataset Info:")
print(df.info())
print("\nMissing Values:")
print(df.isnull().sum())

# Select features and target using exact lowercase names from the dataset
features = ['TV', 'radio', 'newspaper']
target = 'sales'

X = df[features]
y = df[target]

# Pair plot on features and target
sns.pairplot(df)
```

```
plt.suptitle('Pair Plot of Features and Sales', y=1.02)
plt.show()
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
Index: 200 entries, 1 to 200
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
```

3. Split the Dataset

```
from sklearn.model_selection import train_test_split

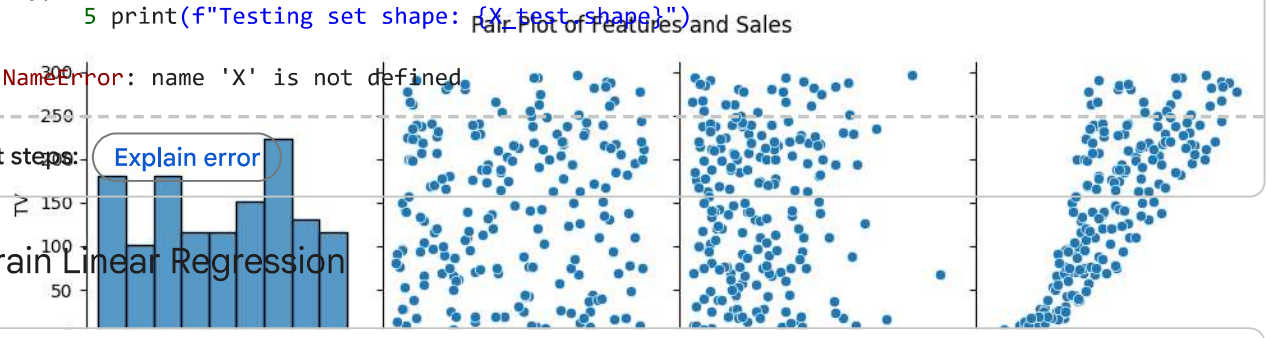
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Training set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
```

```
memory usage: 7.8 KB
None
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_3453/3205407194.py in <cell line: 0>()
      1 from sklearn.model_selection import train_test_split
      2
----> 3 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
      4 print(f"Training set shape: {X_train.shape}")
      5 print(f"Testing set shape: {X_test.shape}")

NameError: name 'X' is not defined
```

Missing values:
TV 0
radio 0
newspaper 0
sales 0
dtype: int64

Next steps: Explain error



4. Train Linear Regression

```
from sklearn.linear_model import LinearRegression

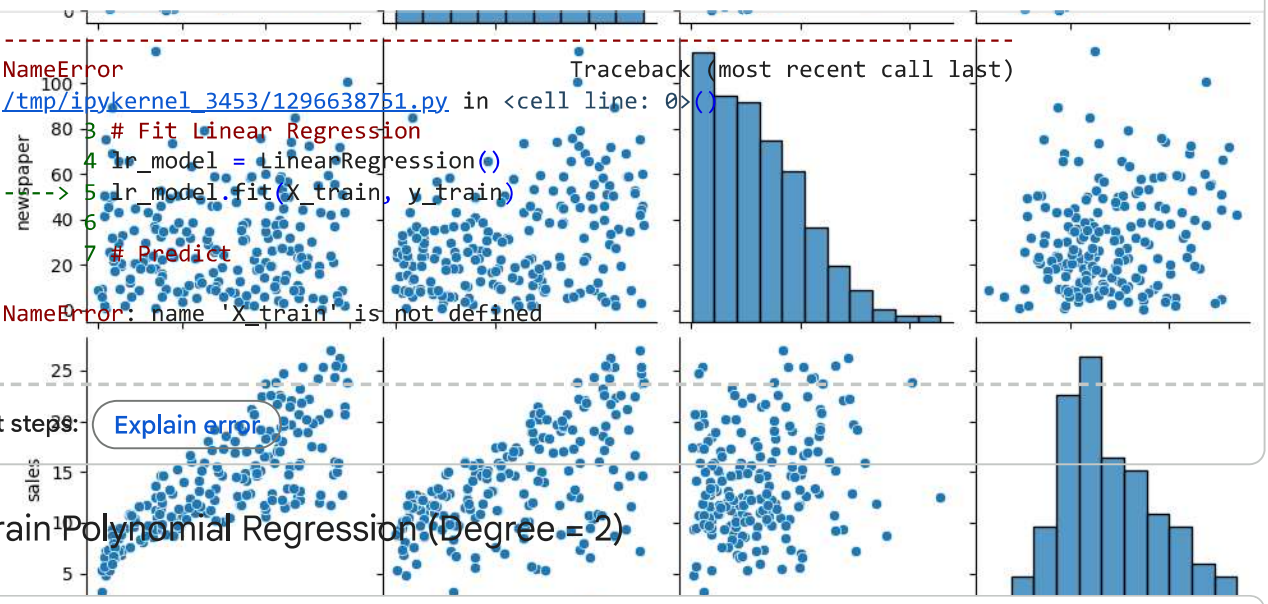
# Fit Linear Regression
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# Predict
y_pred_lr = lr_model.predict(X_test)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_3453/1296638751.py in <cell line: 0>()
      3 # Fit Linear Regression
      4 lr_model = LinearRegression()
----> 5 lr_model.fit(X_train, y_train)
      6
      7 # Predict

NameError: name 'X_train' is not defined
```

Next steps: Explain error



5. Train Polynomial Regression (Degree = 2)

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
```

```
# Create and fit Polynomial Regression Pipeline
poly_model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
poly_model.fit(X_train, y_train)

# Predict
y_pred_poly = poly_model.predict(X_test)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_3453/962877324.py in <cell line: 0>()
      4 # Create and fit Polynomial Regression Pipeline
      5 poly_model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
----> 6 poly_model.fit(X_train, y_train)
      7
      8 # Predict

NameError: name 'X_train' is not defined
```

Next steps: Explain error

6. Train Support Vector Regression (SVR)

We should scale the data first since SVR is sensitive to feature scales.

```
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler

# SVR with RBF kernel within a pipeline to handle feature scaling
svr_model = make_pipeline(StandardScaler(), SVR(kernel='rbf'))
svr_model.fit(X_train, y_train)

# Predict
y_pred_svr = svr_model.predict(X_test)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_3453/2278202515.py in <cell line: 0>()
      4 # SVR with RBF kernel within a pipeline to handle feature scaling
      5 svr_model = make_pipeline(StandardScaler(), SVR(kernel='rbf'))
----> 6 svr_model.fit(X_train, y_train)
      7
      8 # Predict

NameError: name 'X_train' is not defined
```

Next steps: Explain error

7. Evaluate Models and Compare

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

models = {
    'Linear Regression': y_pred_lr,
```

```
    'Polynomial Regression': y_pred_poly,
    'SVR': y_pred_svr
}

metrics_list = []

for name, pred in models.items():
    mae = mean_absolute_error(y_test, pred)
    mse = mean_squared_error(y_test, pred)
    rmse = np.sqrt(mse)
    metrics_list.append({'Model': name, 'MAE': mae, 'MSE': mse, 'RMSE': rmse})

metrics_df = pd.DataFrame(metrics_list)
display(metrics_df)

# Melt dataframe for easy plotting with seaborn
metrics_melted = metrics_df.melt(id_vars='Model', var_name='Metric', value_name='Value')

# Plot a single grouped bar graph comparison
plt.figure(figsize=(10, 6))
sns.barplot(x='Metric', y='Value', hue='Model', data=metrics_melted, palette='viridis')
plt.title('Comparison of Model Performance Metrics')
plt.ylabel('Metric Value')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_3453/2857624539.py in <cell line: 0>()
      2
      3 models = {
----> 4     'Linear Regression': y_pred_lr,
      5     'Polynomial Regression': y_pred_poly,
      6     'SVR': y_pred_svr

NameError: name 'y_pred_lr' is not defined
```

Next steps: Explain error

8. Apply K-Means Clustering

```
from sklearn.cluster import KMeans

# Fit KMeans on features (TV, radio, newspaper)
kmeans = KMeans(n_clusters=3, random_state=42)
df['Cluster'] = kmeans.fit_predict(X)

print("Cluster Assignments:")
display(df.head())
```